Tomasulo Simulator Assignment Report

Ahmed Ahmed

Computer Architecture
October 25 2026

GitHub repository: https://github.com/ahmedshehata08-iss/tomasulo-simulator

1 Assignment implementation

I extended the supplied Python simulator to issue one to four instructions in program order. The implementation follows the programming assignment handout: one CDB, speculative branches with an eight-entry BTB, an unpipelined integer unit, pipelined floating-point units, and a single-ported memory. The report includes before-and-after source snapshots and regenerated timing tables for every issue width.

The main experiments change only issue width. Write-back bandwidth stays at one result per cycle and memory has one nonpipelined port. I use one commit per cycle as a documented design choice because the handout does not specify retirement bandwidth.

The submission passes 67 automated test methods, including 80 seeded sequential-equivalence runs, plus 400 additional mixed-instruction stress runs. All 36 terminating benchmark runs match an independent sequential interpreter. The handout's literal illustrative loop is separately checked for the expected cycle-limit stop.

Report organization

Section	Evidence
2	Machine structure, cycle rules, and assignment requirements
3 to 9	Changes to issue, renaming, execution, branches, memory, and input
10 to 12	Test inputs, hardware configurations, and width comparison
13	Eight complete sample-program timing tables
14 to 16	Branch and memory recovery, validation, and reproduction

Source attribution

The supplied starter identifies Qing as its original author. I preserved that attribution and included the unchanged starter for comparison. The assignment modifications, added tests, and this report are credited to Ahmed Ahmed. The assignment handout defines the required behavior; the sample report guides the presentation and supplies two benchmark instruction sequences.

2 Machine structure and timing

A Simulator object owns the instruction buffer, architectural register files, RAT, reservation stations, ROB, load/store queue, functional-unit activity, and branch predictor. Each dynamic instruction receives a unique increasing identifier; loop iterations and replayed instructions therefore have distinct tags.

Stage	Implemented rule
ISSUE	Fetch and decode a program-order prefix of up to W instructions. Stop at the first unavailable ROB, RS, or LSQ resource. Predict branches here.
EX	Select oldest ready operations. Integer units and dedicated address adders are unpipelined; FP adders and multipliers accept one new operation per unit per cycle.
MEM	One nonpipelined port. Forward an exact matching older store in one cycle or access memory. Track loads that pass unresolved older stores.
WB	One CDB broadcasts one completed register result per cycle. Dependents use the result from the following cycle.
COMMIT	Retire the ready ROB head. Update architectural registers or memory. Branches and stores do not consume the CDB.

An L-cycle operation has end = start + L - 1. An operation may enter its next stage only after its preceding stage ends. The implementation visits COMMIT, WB, MEM, EX, then ISSUE each cycle and applies explicit readiness dates. A unit completing in the current cycle remains occupied for that cycle. CDB arbitration uses completion cycle, then dynamic instruction age.

Requirements and design choices

Handout requirement	Implementation evidence
32 integer and 32 FP registers; R0 fixed	Both register files initialize to zero; renaming never changes R0.
256 bytes and single-precision transfers	A 256-byte array stores four-byte binary32 values; legal starts are byte addresses 0 through 252.
Word branch displacement	Target instruction index = PC index + 1 + offset.
One-bit predictor and 8-entry BTB	One direction bit per static branch, tagged BTB indexed by PC index & 7.
Recovery after EX cycle n	Squash younger work, rebuild the RAT, and resume fetch at n + 2.

3 Code changes for multiple issue

The starter handles one instruction per call to issue. I replaced that dispatch path with a width-bounded loop that allocates a contiguous program-order prefix. When the oldest candidate cannot reserve a resource, the bundle stops; a younger instruction cannot bypass it during issue.

Before starter issue.py

```
178      ins = instructions[PC.PC]
179      op = ins.split(' ')[0]
180      '''decode'''
181      # LD/SD instructions
182      if (op=='Ld')|(op=='Sd'):
183          # check space
184          if (len(ROB)<size_ROB)&(len(ld_sd_queue)<size_ld_sd_queue):
185              # find ldsd_tag
186              ldsd_tag = find_tag_new_ldsd_entry(ld_sd_queue, size_ld_sd_queue)
187              # put ins into ROB
188              put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, ldsd_tag)
189              # put ins into ld_sd_queue
190              put_ins_into_ldsd(ldsd_tag, ld_sd_queue, ins, ROB, rat_int, rat_fp)
191              # update RAT
192              update_rat(ROB, rat_int, rat_fp)
```

Before starter issue.py. Source lines 178-192.

After issue.py

```
 7  def issue(sim):
 8      """Issue a contiguous program-order prefix; a full resource stops the bundle."""
 9      if sim.cycle < sim.fetch_resume_cycle:
10          return
11      for _ in range(sim.config.issue_width):
12          if sim.branch_pending is not None or sim.pc >= len(sim.program):
13              break
14          ins = sim.program[sim.pc]
15          if len(sim.rob) >= sim.config.rob_size:
16              break
17          if ins.kind == "memory":
18              if len(sim.lsq) >= sim.config.lsq_size:
19                  break
20          elif len(sim.stations[ins.kind]) >= sim.rs_capacity[ins.kind]:
21              break
22          # Read every source BEFORE installing the destination rename.
23          operands = [sim.read_operand(name) for name in ins.sources]
24          if ins.op == "Addi":
```

After issue.py. Source lines 7-24.

Source operands are captured before the destination tag is installed. A later instruction in the same bundle observes the new producer tag, while a self-dependent Addi reads its preceding value. Branch prediction selects the next instruction within the same bundle when capacity remains. Issue width defaults to one and accepts only values from one to four.

4 Code changes for renaming and commit

I separated architectural register values from producer tags. Every result wakes only operands waiting for its dynamic identifier. The RAT continues to name the youngest writer, preventing a late older result from replacing a younger dependency.

Before starter wb.py

```
23      dest_tag = cdb.dest_tag
24      index = find_ROB_entry(ROB, dest_tag)
25      reg_tag = ROB[index].dest_tag
26      # rat_int
27      if reg_tag[0] == 'R':
28          rat_int[int(reg_tag[1:])] = cdb.value
29      # rat_fp
30      if reg_tag[0] == 'F':
31          rat_fp[int(reg_tag[1:])] = cdb.value
```

Before starter wb.py. Source lines 23-31.

After wb.py

```
 6  def wb(sim):
 7      eligible = sorted((e for e in sim.results if e.complete_cycle < sim.cycle),
 8                        key=lambda e: (e.complete_cycle, e.id))
 9      for entry in eligible[:sim.config.writeback_width]:
10          entry.wb = sim.cycle
11          entry.status = "writeback"
12          entry.ready_cycle = sim.cycle
13          sim.results.remove(entry)
14          # Broadcast to waiting consumers by producer tag, not by register name.
15          for consumer in sim.rob:
16              for operand in consumer.operands:
17                  if operand.tag == entry.id:
18                      operand.value = entry.result
19                      operand.tag = None
20                      operand.ready_cycle = sim.cycle
21          # RAT remains a mapping to the youngest writer until that writer commits.
```

After wb.py. Source lines 6-21.

After commit.py architectural update

```
19          elif ins.dest is not None and ins.dest != "R0":
20              value = int(entry.result) if ins.dest.startswith("R") else float(entry.result)
21              sim.registers[ins.dest] = value
22              if sim.rat.get(ins.dest) == entry.id:
23                  del sim.rat[ins.dest]
24          entry.commit = sim.cycle
25          entry.status = "committed"
26          sim.rob.popleft()
27          sim.registers["R0"] = 0
```

After commit.py architectural update. Source lines 19-27.

Only the ROB head can retire. A register commit clears its RAT entry only if that entry still belongs to the retiring instruction. Stores update memory at commit and remain in the LSQ until then. A squashed instruction never commits. Tests cover RAW, WAR, WAW, self-renaming, late readers, and recovery of an older surviving producer.

5 Code changes for execution timing

The starter can advance active FU counters twice when another operation is dispatched. I replaced incremental counters with fixed completion dates and separate resource rules for integer, floating-point, and address execution.

Before first counter advancement

```
120     # functional units
121     fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
122     fu_exe(fu_fp_adder, results_buffer, ROB, time_fu_fp_adder, cycle, PC)
123     fu_exe(fu_fp_multi, results_buffer, ROB, time_fu_fp_multi, cycle, PC)
```

Before first counter advancement. Source lines 120-123.

Before repeated counter advancement

```
149     # fp_adder
150     if check_valid_ins_in_rs(rs_fp_adder)>=0:
151         index = check_valid_ins_in_rs(rs_fp_adder)
152         fu_fp_adder.append(fu_entry())
153         add_entry_into_fu(fu_fp_adder, rs_fp_adder[index])
154         if ROB[find_ROB_entry(ROB, rs_fp_adder[index].dest_tag)].issue[0] < cycle:
155             fu_exe(fu_fp_adder, results_buffer, ROB, time_fu_fp_adder, cycle, PC)
156         # remove ins from rs
```

Before repeated counter advancement. Source lines 149-156.

After fixed start and completion dates

```
 6 def _start(sim, entry):
 7     ins = entry.instruction
 8     entry.status = "executing"
 9     entry.exe_start = sim.cycle
10     latency = sim.latencies[ins.kind]
11     entry.exe_end = sim.cycle + latency - 1
12     if ins.kind == "memory":
13         # A store can compute its address while its data is still pending.
14         entry.address = int(entry.operands[0].value) + ins.immediate
15         if not 0 <= entry.address <= sim.config.memory_size - 4:
```

After fixed start and completion dates. Source lines 6-15.

After unpipelined integer occupancy

```
35 def exe(sim):
36     # FP pipelines accept one operation per unit per cycle; integer units do not.
37     slots = dict(sim.unit_counts)
38     slots["int"] -= sum(entry.instruction.kind == "int" for entry in sim.arithmetic_active)
39     slots["memory"] = sim.config.addr_units - len(sim.address_active)
40     waiting = sorted(
41         [e for station in sim.stations.values() for e in station]
42         + [e for e in sim.lsq if e.exe_start is None], key=lambda e: e.id
```

After unpipelined integer occupancy. Source lines 35-42.

Integer occupancy is subtracted from available units, so a three-cycle integer operation blocks its unit for all three cycles. Floating-point units instead use an initiation interval of one cycle. Load/store address calculation uses dedicated adders and never occupies an integer ALU. The benchmark integer_unpipelined exercises latency three, while separate tests verify overlapping FP operations.

6 Code changes for branch prediction

The starter disables further issue at Bne until its result is known. I added prediction during issue, an eight-entry direct-mapped target buffer with PC tags, and a separate one-bit direction state for each static branch. A cold branch predicts not taken; a taken prediction needs a matching BTB tag.

Before starter branch issue stalls the PC

```
234     # Bne
235     elif (op=='Bne'):
236         # check space
237         if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
238             # put ins into ROB
239             put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
240             # put ins into rs_int_adder
241             index = check_rs_space(rs_int_adder)
242             put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
243             # PC
244             PC.valid = 0
```

Before starter branch issue stalls the PC. Source lines 234-244.

After prediction during ISSUE

```
85      def predict_branch(self, entry):
86          """Predict during ISSUE; a taken prediction requires a tagged BTB hit."""
87          index = entry.pc & 7
88          target = self.btb[index]
89          hit = target is not None and target["pc"] == entry.pc
90          taken = bool(self.predictor.get(entry.pc, False) and hit)
91          entry.predicted_taken = taken
92          entry.predicted_next_pc = target["target"] if taken else entry.pc + 1
93          self.branch_predictions.append({
94              "id": entry.id, "pc": entry.pc, "cycle": self.cycle,
95              "btb_index": index, "btb_hit": hit, "predicted_taken": taken,
96              "predicted_next_pc": entry.predicted_next_pc,
97          })
98          return entry.predicted_next_pc
99
```

After prediction during ISSUE. Source lines 85-99.

The BTB index is the low three bits of the word PC, implemented as instruction_index & 7. Two branches that share an index replace the target entry, and the stored tag prevents using the other branch's target. At the end of EX, the resolved outcome trains the direction bit and target entry. Both Beq and Bne use integer reservation stations.

7 Code changes for recovery

A misprediction removes every younger instruction from the ROB, RS, LSQ, active units, and pending result list. Wrong-path operand wait tags are cleared. Architectural state remains precise because only in-order commit updates registers and memory.

After remove the wrong path and rebuild RAT

```
116         self.rob = deque(entry for entry in self.rob if entry.id not in ids)
117         self.lsq[:] = [entry for entry in self.lsq if entry.id not in ids]
118         for station in self.stations.values():
119             station[:] = [entry for entry in station if entry.id not in ids]
120         for active in (self.arithmetic_active, self.address_active, self.memory_active, self.results):
121             active[:] = [entry for entry in active if entry.id not in ids]
122         self.rat.clear()
123         for entry in self.rob:
124             destination = entry.instruction.dest
125             if destination is not None and destination != "R0":
126                 self.rat[destination] = entry.id
127         self.branch_pending = None
128         self.pc = target_pc
129         # The cycle after detection performs recovery; fetch follows it.
130         self.fetch_resume_cycle = self.cycle + 2
131         return sorted(ids)
132 
133     def resolve_branch(self, entry):
```

After remove the wrong path and rebuild RAT. Source lines 116-133.

After misprediction event and redirect

```
143           entry.mispredicted = actual_next != entry.predicted_next_pc
144           if entry.mispredicted:
145               discarded = self.squash(entry.id + 1, actual_next,
146                                       f"branch misprediction at instruction {entry.id}")
147               self.branch_mispredictions.append({
148                   "id": entry.id, "pc": entry.pc, "cycle": self.cycle,
149                   "predicted_next_pc": entry.predicted_next_pc,
150                   "actual_next_pc": actual_next, "target": actual_next,
151                   "resume_cycle": self.fetch_resume_cycle, "squashed_ids": discarded,
152               })
```

After misprediction event and redirect. Source lines 143-152.

Rebuilding the RAT from the surviving ROB entries restores the youngest valid producer for every destination. The fetch resume cycle is detection cycle + 2: the intervening cycle performs recovery. Historical trace rows remain visible with status squashed; an interrupted stage shows X instead of a completion cycle. IPC counts committed instructions only.

Memory-order violations use the same recovery machinery, starting with the oldest invalid load rather than the instruction after a branch. This replay policy is an implementation choice because the handout permits speculative loads but does not prescribe violation recovery timing.

8 Code changes for memory

The starter examines a preceding store without a complete age-ordered search. I check older stores, forward the youngest relevant exact match in one cycle, and allow a load to access memory when older store addresses remain unresolved, as the handout specifies.

Before starter store check

```
15 from init import fu_result
16
17 # function: forward check from sd
18 # only check nearest sd
19 def forward_check_from_sd(ld_sd_queue, index):
20     data = []
21     for i in reversed(range(index)):
22         if (ld_sd_queue[i].op=='Sd'):
23             if (ld_sd_queue[i].address == ld_sd_queue[index].address):
24                 if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
25                     data.append(ld_sd_queue[i].data)
26             break
27     return data
```

Before starter store check. Source lines 15-27.

After select a load source

```
 6 def load_source(sim, load):
 7     """Return (eligible, value, forwarding id), checking all older stores."""
 8     older = [e for e in sim.lsq if e.id < load.id and e.instruction.op == "Sd"]
 9     unknown = [e for e in older if e.exe_end is None or e.exe_end >= sim.cycle]
10     known = [e for e in older if e.exe_end is not None and e.exe_end < sim.cycle]
11     matches = [e for e in known if abs(e.address - load.address) < 4]
12     if matches:
13         source = matches[-1]  # The youngest overlapping store determines the source.
14         # A partial overlap needs bytes from memory and stores: wait for commit.
15         # An exact match overwrites all four bytes and can forward directly.
16         if source.address != load.address:
17             return False, None, None
18         if not source.operands[1].ready(sim.cycle):
19             return False, None, None
20         load.unresolved_store_ids = [e.id for e in unknown]
21         return True, sim.memory.normalize(source.operands[1].value), source.id
22     # The handout permits access past unresolved stores; EX later checks aliases.
23     load.unresolved_store_ids = [e.id for e in unknown]
24     return True, sim.memory.get(load.address, 0.0), None
25
```

After select a load source. Source lines 6-25.

A load records unresolved older-store identifiers. If a later address calculation reveals an overlapping access, the load and its younger instructions are squashed and fetched again. An exact younger forwarding source can make an even older store irrelevant. Known partial-byte overlaps wait for commit so the load reads the correct combined bytes. Completed loads release their LSQ entries immediately.

Memory is a real 256-byte array. Loads, stores, and forwarded values use binary32 rounding. Unaligned addresses are supported because the handout's own input computes address 18. Overlapping accesses share bytes; initialized and stored address views plus nonzero aligned words appear in the memory output.

9 Code changes for assignment input

The new primary interface reads configuration, initial state, and assembly from one case-insensitive text file. Hardware rows specify reservation-station counts, execution latency, memory latency where applicable, and unit counts. The load/store row configures address units; memory itself remains a single port in all assignment benchmarks.

Before fixed-position address parsing

```
141      ld_sd_queue[-1].dest_tag = ROB[-1].ROB_tag
142      # immediate and reg value
143      ld_sd_queue[-1].immediate = int(ins.split(' ')[-1][:-4])
144      register = ins.split(' ')[-1][-3:-1] # register name
145      if (type(rat_int[int(register[1:])]) == str):
146          ld_sd_queue[-1].reg_tag = rat_int[int(register[1:])]
147          ld_sd_queue[-1].valid = 0
148      else:
149          ld_sd_queue[-1].reg_value = rat_int[int(register[1:])]
150          ld_sd_queue[-1].valid = 1
```

Before fixed-position address parsing. Source lines 141-150.

After one-file input parser

```
247  def parse_input(text):
248      """Read the handout's single-file hardware, initial state, and assembly format.
249
250      The load/store row gives LSQ entries, address EX latency, memory latency,
251      and address-unit count. Memory itself remains single-ported. Numeric branch
252      displacements are in words; data addresses are byte addresses. Return
253      (program, Config, initial_registers, initial_memory).
254      """
255      if not isinstance(text, str):
256          raise ValueError("input must be text")
257      values, registers, memory, assembly = {}, {}, {}, []
258      row_fields = {
259          "integer adder": ("rs_int", "int_latency", "int_units"),
260          "fp adder": ("rs_fp_add", "fp_add_latency", "fp_add_units"),
261          "fp multiplier": ("rs_fp_mul", "fp_mul_latency", "fp_mul_units"),
262          "load/store unit": ("lsq_size", "addr_latency", "mem_latency", "addr_units"),
263      }
264      config_names = {f.name.replace("_", " "): f.name for f in fields(Config)}
```

After one-file input parser. Source lines 247-264.

After word-based branch displacement

```
220                      target = pc + 1 + offset
221                  if not 0 <= target <= len(lines):
222                      raise ValueError("branch target is outside the program")
223                  ins = Instruction(op, line, sources=sources, immediate=offset, target=target)
```

After word-based branch displacement. Source lines 220-223.

Registers omitted from the input are zero. The parser accepts commas, whitespace, comments, signed offsets, multi-digit registers, and labels. It rejects malformed hardware settings, invalid registers, out-of-range memory addresses, and branch targets outside the program. The legacy assembly-plus-JSON interface remains available as an optional extension; the examples and report use --input.

10 Test case 1 configuration and input

This test uses the instruction sequence from sample report case 1. The assignment profile holds the CDB and memory port counts at one for every issue width. All unlisted registers are zero; R1 = 12, R2 = 32, and F20 = 3.0 initially.

Unit	RS or LSQ	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	4	3	-	1
FP multiplier	4	5	-	1
Load and store	4	1	6	1

ROB entries = 64. Integer units and address adders are unpipelined; FP units are pipelined. The load/store FU count refers to address adders. Memory has one nonpipelined port.

Address	4	8	12	24	28	32
Value	3	2	1	6	5	4

```
 1. Ld F1 0(R1)
 2. Mult.d F2 F1 F0
 3. Add.d F3 F3 F2
 4. Mult.d F2 F1 F1
 5. Add.d F4 F2 F4
 6. Addi R1 R1 8
 7. Ld F1 0(R1)
 8. Mult.d F2 F1 F0
 9. Add.d F3 F3 F2
10. Mult.d F2 F1 F1
11. Add.d F4 F2 F4
12. Addi R1 R1 8
13. Ld F1 0(R1)
14. Mult.d F2 F1 F0
15. Add.d F3 F3 F2
16. Mult.d F2 F1 F1
17. Add.d F4 F2 F4
```

Expected final values: R1 = 28, F1 = 5, F2 = 25, F3 = 0, F4 = 26.

The three loads read 1, 0, and 5. F4 accumulates their squares, 1 + 0 + 25 = 26. F0 is zero, so F3 stays zero.

11 Test case 2 configuration and input

This test uses the instruction sequence from sample report case 2. The assignment profile holds the CDB and memory port counts at one for every issue width. All unlisted registers are zero; R1 = 12, R2 = 32, and F20 = 3.0 initially.

Unit	RS or LSQ	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	3	3	-	2
FP multiplier	2	15	-	2
Load and store	4	1	5	2

ROB entries = 128. Integer units and address adders are unpipelined; FP units are pipelined. The load/store FU count refers to address adders. Memory has one nonpipelined port.

Address	4	8	12	24	28	32
Value	3	2	1	6	5	4

```
 1. Ld F1 0(R1)
 2. Mult.d F2 F1 F0
 3. Add.d F3 F3 F2
 4. Mult.d F2 F1 F1
 5. Ld F1 0(R1)
 6. Addi R1 R1 4
 7. Add.d F4 F2 F4
 8. Addi R1 R1 8
 9. Ld F1 0(R1)
10. Mult.d F2 F1 F0
11. Add.d F3 F3 F2
12. Mult.d F2 F1 F1
13. Ld F1 0(R1)
14. Mult.d F2 F1 F0
15. Add.d F3 F3 F2
16. Mult.d F2 F1 F1
17. Add.d F4 F2 F4
```

Expected final values: R1 = 24, F1 = 6, F2 = 36, F3 = 0, F4 = 37.

The early loads read 1 and the later loads read 6. F4 becomes 1 + 36 = 37. The sample's two memory units are adapted to the handout's single memory port, so its published cycle totals are not acceptance targets.

12 Results across issue widths

Each width is a fresh run with the same program and initial state. Only issue width changes. CDB width = 1, commit width = 1, memory ports = 1, and branch prediction is enabled. Every run verifies all registers, final memory, and committed instruction count against an independent sequential interpreter.

Program	W1	W2	W3	W4	W4 speedup
Sample case 1	36	36	36	36	1.000x
Sample case 2	47	46	46	46	1.022x
Starter branch	45	45	45	45	1.000x
Complete loop	67	66	66	66	1.015x
Independent integer	27	27	27	27	1.000x
Terminating handout	10	9	9	9	1.111x
Branch recovery	28	25	25	25	1.120x
Memory replay	19	19	19	19	1.000x
Three cycle integer	12	12	12	12	1.000x

The table reports total cycles, including pipeline fill, recovery, and drain. IPC is committed instructions divided by cycles. Speedup is width-1 cycles divided by the compared width's cycles. Squashed instructions consume resources but do not count as committed work.

Interpretation

Sample case 1 takes 36, 36, 36, 36 cycles at widths 1 through 4. Sample case 2 takes 47, 46, 46, 46 cycles. A wider issue stage cannot remove dependency delays, the one-result CDB limit, or single-port memory occupancy. For some short programs, earlier speculative work also changes resource contention and recovery timing.

Relationship to the sample report

The sample is a formatting reference, not a timing oracle for this machine. Its wider-issue comparisons also widen write back and commit, and its second configuration has two memory units. This report keeps the handout's single CDB and single memory port. All dependencies obey one consistent rule: a WB value is usable in the following cycle. Matching a total cycle count would not establish identical intermediate timings.

The unchanged literal handout example loops because R2 = 20 and R3 = 0 never change. It is retained as handout_literal.in and reaches the configured cycle limit at all four widths. assignment_demo.in changes only R3 to 20 so the example terminates; that adaptation is labeled explicitly.

13 Detailed instruction timings

Test case 1 with issue width 1

Issue width = 1. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-8	9	10
2	Mult.d F2 F1 F0	2	10-14	-	15	16
3	Add.d F3 F3 F2	3	16-18	-	19	20
4	Mult.d F2 F1 F1	4	11-15	-	16	21
5	Add.d F4 F2 F4	5	17-19	-	20	22
6	Addi R1 R1 8	6	7-7	-	8	23
7	Ld F1 0(R1)	7	9-9	10-15	17	24
8	Mult.d F2 F1 F0	8	18-22	-	23	25
9	Add.d F3 F3 F2	9	24-26	-	27	28
10	Mult.d F2 F1 F1	10	19-23	-	24	29
11	Add.d F4 F2 F4	11	25-27	-	28	30
12	Addi R1 R1 8	12	13-13	-	14	31
13	Ld F1 0(R1)	13	15-15	16-21	22	32
14	Mult.d F2 F1 F0	14	23-27	-	29	33
15	Add.d F3 F3 F2	16	30-32	-	33	34
16	Mult.d F2 F1 F1	17	24-28	-	30	35
17	Add.d F4 F2 F4	18	31-33	-	34	36

Figure 1. Generated timing table for test case 1, width 1.

The run commits 17 instructions in 36 cycles. IPC = 0.472; speedup relative to width 1 = 1.000x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 10-14 and 11-15. Their WB cycles are 15 and 16. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	28	5.0	25.0	0.0	26.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 1 with issue width 2

Issue width = 2. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-8	9	10
2	Mult.d F2 F1 F0	1	10-14	-	15	16
3	Add.d F3 F3 F2	2	16-18	-	19	20
4	Mult.d F2 F1 F1	2	11-15	-	17	21
5	Add.d F4 F2 F4	3	18-20	-	21	22
6	Addi R1 R1 8	3	4-4	-	5	23
7	Ld F1 0(R1)	4	6-6	9-14	16	24
8	Mult.d F2 F1 F0	4	17-21	-	23	25
9	Add.d F3 F3 F2	5	24-26	-	27	28
10	Mult.d F2 F1 F1	5	18-22	-	24	29
11	Add.d F4 F2 F4	6	25-27	-	28	30
12	Addi R1 R1 8	6	7-7	-	8	31
13	Ld F1 0(R1)	7	9-9	15-20	22	32
14	Mult.d F2 F1 F0	10	23-27	-	29	33
15	Add.d F3 F3 F2	16	30-32	-	33	34
16	Mult.d F2 F1 F1	16	24-28	-	30	35
17	Add.d F4 F2 F4	18	31-33	-	34	36

Figure 2. Generated timing table for test case 1, width 2.

The run commits 17 instructions in 36 cycles. IPC = 0.472; speedup relative to width 1 = 1.000x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 10-14 and 11-15. Their WB cycles are 15 and 17. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	28	5.0	25.0	0.0	26.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 1 with issue width 3

Issue width = 3. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-8	9	10
2	Mult.d F2 F1 F0	1	10-14	-	15	16
3	Add.d F3 F3 F2	1	16-18	-	19	20
4	Mult.d F2 F1 F1	2	11-15	-	17	21
5	Add.d F4 F2 F4	2	18-20	-	21	22
6	Addi R1 R1 8	2	3-3	-	4	23
7	Ld F1 0(R1)	3	5-5	9-14	16	24
8	Mult.d F2 F1 F0	3	17-21	-	23	25
9	Add.d F3 F3 F2	3	24-26	-	27	28
10	Mult.d F2 F1 F1	4	18-22	-	24	29
11	Add.d F4 F2 F4	4	25-27	-	28	30
12	Addi R1 R1 8	4	5-5	-	6	31
13	Ld F1 0(R1)	5	7-7	15-20	22	32
14	Mult.d F2 F1 F0	10	23-27	-	29	33
15	Add.d F3 F3 F2	16	30-32	-	33	34
16	Mult.d F2 F1 F1	16	24-28	-	30	35
17	Add.d F4 F2 F4	18	31-33	-	34	36

Figure 3. Generated timing table for test case 1, width 3.

The run commits 17 instructions in 36 cycles. IPC = 0.472; speedup relative to width 1 = 1.000x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 10-14 and 11-15. Their WB cycles are 15 and 17. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	28	5.0	25.0	0.0	26.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 1 with issue width 4

Issue width = 4. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-8	9	10
2	Mult.d F2 F1 F0	1	10-14	-	15	16
3	Add.d F3 F3 F2	1	16-18	-	19	20
4	Mult.d F2 F1 F1	1	11-15	-	17	21
5	Add.d F4 F2 F4	2	18-20	-	21	22
6	Addi R1 R1 8	2	3-3	-	4	23
7	Ld F1 0(R1)	2	5-5	9-14	16	24
8	Mult.d F2 F1 F0	2	17-21	-	23	25
9	Add.d F3 F3 F2	3	24-26	-	27	28
10	Mult.d F2 F1 F1	3	18-22	-	24	29
11	Add.d F4 F2 F4	3	25-27	-	28	30
12	Addi R1 R1 8	3	5-5	-	6	31
13	Ld F1 0(R1)	4	7-7	15-20	22	32
14	Mult.d F2 F1 F0	10	23-27	-	29	33
15	Add.d F3 F3 F2	16	30-32	-	33	34
16	Mult.d F2 F1 F1	16	24-28	-	30	35
17	Add.d F4 F2 F4	18	31-33	-	34	36

Figure 4. Generated timing table for test case 1, width 4.

The run commits 17 instructions in 36 cycles. IPC = 0.472; speedup relative to width 1 = 1.000x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 10-14 and 11-15. Their WB cycles are 15 and 17. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	28	5.0	25.0	0.0	26.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 2 with issue width 1

Issue width = 1. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-7	8	9
2	Mult.d F2 F1 F0	2	9-23	-	24	25
3	Add.d F3 F3 F2	3	25-27	-	28	29
4	Mult.d F2 F1 F1	4	9-23	-	25	30
5	Ld F1 0(R1)	5	6-6	8-12	13	31
6	Addi R1 R1 4	6	7-7	-	9	32
7	Add.d F4 F2 F4	7	26-28	-	29	33
8	Addi R1 R1 8	8	10-10	-	11	34
9	Ld F1 0(R1)	9	12-12	13-17	18	35
10	Mult.d F2 F1 F0	10	19-33	-	34	36
11	Add.d F3 F3 F2	11	35-37	-	38	39
12	Mult.d F2 F1 F1	12	19-33	-	35	40
13	Ld F1 0(R1)	13	14-14	18-22	23	41
14	Mult.d F2 F1 F0	19	24-38	-	39	42
15	Add.d F3 F3 F2	25	40-42	-	43	44
16	Mult.d F2 F1 F1	26	27-41	-	42	45
17	Add.d F4 F2 F4	27	43-45	-	46	47

Figure 5. Generated timing table for test case 2, width 1.

The run commits 17 instructions in 47 cycles. IPC = 0.362; speedup relative to width 1 = 1.000x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 9-23 and 9-23. Their WB cycles are 24 and 25. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	24	6.0	36.0	0.0	37.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 2 with issue width 2

Issue width = 2. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-7	8	9
2	Mult.d F2 F1 F0	1	9-23	-	24	25
3	Add.d F3 F3 F2	2	25-27	-	28	29
4	Mult.d F2 F1 F1	2	9-23	-	25	30
5	Ld F1 0(R1)	3	4-4	8-12	13	31
6	Addi R1 R1 4	3	4-4	-	5	32
7	Add.d F4 F2 F4	4	26-28	-	29	33
8	Addi R1 R1 8	4	6-6	-	7	34
9	Ld F1 0(R1)	5	8-8	13-17	18	35
10	Mult.d F2 F1 F0	9	19-33	-	34	36
11	Add.d F3 F3 F2	9	35-37	-	38	39
12	Mult.d F2 F1 F1	10	19-33	-	35	40
13	Ld F1 0(R1)	10	11-11	18-22	23	41
14	Mult.d F2 F1 F0	19	24-38	-	39	42
15	Add.d F3 F3 F2	25	40-42	-	43	44
16	Mult.d F2 F1 F1	25	26-40	-	41	45
17	Add.d F4 F2 F4	26	42-44	-	45	46

Figure 6. Generated timing table for test case 2, width 2.

The run commits 17 instructions in 46 cycles. IPC = 0.370; speedup relative to width 1 = 1.022x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 9-23 and 9-23. Their WB cycles are 24 and 25. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	24	6.0	36.0	0.0	37.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 2 with issue width 3

Issue width = 3. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-7	8	9
2	Mult.d F2 F1 F0	1	9-23	-	24	25
3	Add.d F3 F3 F2	1	25-27	-	28	29
4	Mult.d F2 F1 F1	2	9-23	-	25	30
5	Ld F1 0(R1)	2	3-3	8-12	13	31
6	Addi R1 R1 4	2	3-3	-	4	32
7	Add.d F4 F2 F4	3	26-28	-	29	33
8	Addi R1 R1 8	3	5-5	-	6	34
9	Ld F1 0(R1)	3	7-7	13-17	18	35
10	Mult.d F2 F1 F0	9	19-33	-	34	36
11	Add.d F3 F3 F2	9	35-37	-	38	39
12	Mult.d F2 F1 F1	9	19-33	-	35	40
13	Ld F1 0(R1)	10	11-11	18-22	23	41
14	Mult.d F2 F1 F0	19	24-38	-	39	42
15	Add.d F3 F3 F2	25	40-42	-	43	44
16	Mult.d F2 F1 F1	25	26-40	-	41	45
17	Add.d F4 F2 F4	26	42-44	-	45	46

Figure 7. Generated timing table for test case 2, width 3.

The run commits 17 instructions in 46 cycles. IPC = 0.370; speedup relative to width 1 = 1.022x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 9-23 and 9-23. Their WB cycles are 24 and 25. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	24	6.0	36.0	0.0	37.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

Test case 2 with issue width 4

Issue width = 4. One CDB, one commit per cycle, and one nonpipelined memory port. Hardware and initial state are listed in the corresponding test-case section. EX and MEM ranges are inclusive.

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT
1	Ld F1 0(R1)	1	2-2	3-7	8	9
2	Mult.d F2 F1 F0	1	9-23	-	24	25
3	Add.d F3 F3 F2	1	25-27	-	28	29
4	Mult.d F2 F1 F1	1	9-23	-	25	30
5	Ld F1 0(R1)	2	3-3	8-12	13	31
6	Addi R1 R1 4	2	3-3	-	4	32
7	Add.d F4 F2 F4	2	26-28	-	29	33
8	Addi R1 R1 8	2	5-5	-	6	34
9	Ld F1 0(R1)	3	7-7	13-17	18	35
10	Mult.d F2 F1 F0	9	19-33	-	34	36
11	Add.d F3 F3 F2	9	35-37	-	38	39
12	Mult.d F2 F1 F1	9	19-33	-	35	40
13	Ld F1 0(R1)	9	10-10	18-22	23	41
14	Mult.d F2 F1 F0	19	24-38	-	39	42
15	Add.d F3 F3 F2	25	40-42	-	43	44
16	Mult.d F2 F1 F1	25	26-40	-	41	45
17	Add.d F4 F2 F4	26	42-44	-	45	46

Figure 8. Generated timing table for test case 2, width 4.

The run commits 17 instructions in 46 cycles. IPC = 0.370; speedup relative to width 1 = 1.022x. The independent final-state check passes.

Multiplier example: instructions 2 and 4 execute in cycles 9-23 and 9-23. Their WB cycles are 24 and 25. FP pipeline availability controls execution starts; the single CDB separately controls result broadcast.

Register	R1	F1	F2	F3	F4
Final value	24	6.0	36.0	0.0	37.0

The complete output files also list all 64 architectural registers and nonzero memory. Memory is unchanged for this program.

14 Branch prediction and recovery evidence

This additional terminating loop exercises cold prediction, a learned taken prediction, and a not-taken loop exit. The table records all dynamic instructions, including work that was later squashed. C means committed; X means squashed. A stage range ending in X was interrupted.

```
Addi R1 R0 3
Addi R2 R0 0
Addi R2 R2 4
Addi R1 R1 -1
Bne R1 R0 -3
Addi R3 R0 99
Sd F20 0(R2)
```

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT	State
1	Addi R1 R0 3	1	2-2	-	3	4	C
2	Addi R2 R0 0	1	3-3	-	4	5	C
3	Addi R2 R2 4	1	5-5	-	6	7	C
4	Addi R1 R1 -1	1	4-4	-	5	8	C
5	Bne R1 R0 -3	2	6-6	-	-	9	C
6	Addi R3 R0 99	3	-	-	-	-	X
7	Sd F20 0(R2)	3	-	-	-	-	X
8	Addi R2 R2 4	8	9-9	-	10	11	C
9	Addi R1 R1 -1	8	10-10	-	11	12	C
10	Bne R1 R0 -3	8	12-12	-	-	13	C
11	Addi R2 R2 4	8	11-11	-	12	14	C
12	Addi R1 R1 -1	9	13-13	-	14	15	C
13	Bne R1 R0 -3	10	15-15	-	-	16	C
14	Addi R2 R2 4	11	14-14	-	15	-	X
15	Addi R1 R1 -1	12	-	-	-	-	X
16	Bne R1 R0 -3	13	-	-	-	-	X
17	Addi R2 R2 4	14	-	-	-	-	X
18	Addi R3 R0 99	17	18-18	-	19	20	C
19	Sd F20 0(R2)	17	18-18	20-24	-	25	C

Branch ID	EX resolves	Correct fetch	Discarded IDs
5	6	8	6, 7
13	15	17	14, 15, 16, 17

This width-4 run commits 13 instructions in 25 cycles, with 2 branch mispredictions. For every event, correct fetching resumes at resolution + 2. The regression suite separately verifies BTB index collisions, older RAT recovery, and suppression of wrong-path stores and invalid-address loads.

15 Speculative memory and validation

The load reads byte address 16 because R2 is initialized to 16. The older store also targets 16 after Addi produces R1. With R2 = 0, the load would read address 0 and no replay would occur. This width-4 trace uses one CDB, one commit per cycle, one memory port, and 256-byte memory; all unlisted registers and memory bytes start at zero.

```
# Unit                RS              EX              MEM        FUs
Integer adder          4               4                          1
FP adder               3               3                          1
FP multiplier          2              15                          1
Load/store unit        4               1               2          1
ROB entries = 64
Issue width = 4
R2=16, F20=7
Mem[16]=1
Addi R1 R0 16
Sd F20 0(R1)
Ld F2 0(R2)
Add.d F4 F2 F2
Addi R3 R0 99
```

Reproduce the displayed run with the supplied input file and this issue-width override:

```
python3 code/main.py --input examples/memory_replay.in --issue-width 4
```

ID	Instruction	ISSUE	EX	MEM	WB	COMMIT	State
1	Addi R1 R0 16	1	2-5	-	6	7	C
2	Sd F20 0(R1)	1	7-7	8-9	-	10	C
3	Ld F2 0(R2)	1	2-2	3-4	5	-	X
4	Add.d F4 F2 F2	1	6-X	-	-	-	X
5	Addi R3 R0 99	2	6-X	-	-	-	X
6	Ld F2 0(R2)	9	10-10	11-12	13	14	C
7	Add.d F4 F2 F2	9	14-16	-	17	18	C
8	Addi R3 R0 99	9	10-13	-	14	19	C

Store ID	Invalid load	Detection	Resume
2	3	7	9

Integer EX latency 4 delays the store address until cycle 7. Memory latency 2 lets the original load read in cycles 3-4. The alias triggers replay at 7 and fetching resumes at 9. The run finishes in 19 cycles with F2 = 7, F4 = 14, and Mem[16] = 7. C means committed; X means squashed or an interrupted stage.

Validation coverage

All 67 test methods pass, including 80 seeded equivalence runs. Another 400 stress runs and all 36 terminating benchmark runs pass; four literal handout runs stop at the expected cycle limit. Independent interpreters check architectural state, and a separate checker verifies all 136 main timing rows. Full logs and traces accompany the code.

16 Running the submission and model limits

The package requires Python 3.10 or newer and uses only the standard library for simulation and tests. Run these commands from the extracted Ahmed_Ahmed_Tomasulo directory.

```
python3 code/main.py --input examples/assignment_demo.in
python3 code/main.py --input examples/sample_case1.in --issue-width 4
python3 -m unittest discover -s tests -v
python3 tests/stress_equivalence.py
python3 run_experiments.py
```

For machine-readable output, add --json output.json or --trace-csv timings.csv. The terminal output contains the five-stage instruction table, status of squashed work, both register files, and nonzero memory. README.md explains the one-file format and all parameters.

Model limits and explicit assumptions

The benchmark configuration is the assignment machine. Optional parameters can widen CDB or commit bandwidth, add memory ports, disable prediction, or restrict execution order; these are extensions and are not used in the assignment result tables. Commit width one, cold not-taken prediction, tagged direct-mapped BTB replacement, end-EX training, oldest-ready scheduling, and memory replay at detection + 2 are documented choices where the handout leaves detail open.

Floating-point register arithmetic uses Python binary64 values; memory transfers use IEEE binary32. The simulator does not model caches, interrupts, integer overflow flags, floating-point exception flags, NaN payloads, or a configurable floating-point rounding mode. Unaligned byte accesses are supported with little-endian storage. Known partial-overlap stores may delay a load until commit; exact-address forwarding still takes one cycle.

No failing functional test remains in the submitted suite. Nonterminating programs stop at --max-cycles rather than producing a false final result. Out-of-range effective addresses raise an error only if the instruction reaches commit; wrong-path faults are safely discarded.

Files and submission status

The archive contains the revised code, combined-input test cases, README, generated results, automated tests, before-and-after patch, unchanged original source, and this report. The latest code and report are available in the GitHub repository below.

GitHub repository: https://github.com/ahmedshehata08-iss/tomasulo-simulator

Source material

Programming assignment handout, programming-tomasulo.pdf, four physical pages: processor and LSQ rules on page 1; recovery, hardware, and ISA on page 2; input and output on page 3; tests and submission requirements on page 4. Sample_Tomasulo_Assignment_Report(2).pdf supplies the presentation reference and two instruction sequences. Original source: supplied tomasulo-python.zip, retained unchanged in original/tomasulo-python/.